

Lab Index for Lab Sheet 1

SN	Title	Lab Date	Signature
1.	Write a program that uses two namespaces to calculate salary for "Permanent" and "Contract" employees differently, and display the result neatly formatted using stream manipulators. <i>Permanent::calculateSalary</i> $\rightarrow$ <i>basic + 20% HRA - 10% tax</i> <i>Contract::calculateSalary</i> $\rightarrow$ <i>basic + 10% allowance (no tax)</i> <i>In main(), ask the user for the employee type and basic salary, call the matching namespace's function, and print a neatly formatted salary slip using setw, setfill, fixed, and setprecision(2) so amounts line up in two decimal places.</i>	2083-03-09	
2.	Write a program that ask the user how many students are in a class, use new to dynamically allocate an integer array of that size, and read in marks. Compute and display the highest, lowest, average, and number of students who passed (≥ 40). Then ask if the teacher wants to add more students: dynamically allocate a new, larger array, copy the old data into it, delete[] the old array, and continue. Finally delete[] the last array before exiting.	2083-03-09	
3.	Write a program with overloaded area() functions (using inline for one and a default argument for another) that calculates the area of a square, rectangle, or triangle based on the number of arguments passed. <i>[inline double area(double side) $\rightarrow$ square (1 parameter)</i> <i>double area(double length, double width) $\rightarrow$ rectangle (2 parameters)</i> <i>double area(int base, double int, double factor = 0.5) $\rightarrow$ triangle (3 parameters; factor defaults to 0.5)]</i>	2083-03-09	
4.	Write a program that swaps two integers using a pointer-based function and a reference-based function, and shows why each does or doesn't change the original values in main(). <i>[swapByValue(int a, int b): ordinary value parameters</i> <i>swapByPointer(int *a, int *b): pointers are passed by value, but dereferencing them swaps the actual caller variables</i> <i>swapByReference(int &a, int &b): reference parameters]</i>	2083-03-09	
5.	Write a program that passes an array to a function to find its sum, maximum, and minimum, and another function to check whether the array is sorted.	2083-03-09	

Lab Index for Lab Sheet 2

SN	Title	Lab Date	Signature
1.	Write a program that defines a class "Student" with private data members for roll number, name, and marks obtained in 5 subjects. Use a parameterized constructor to initialize the roll number and name, and a member function to input the marks. Compute the total, percentage, and grade, then display the complete result.	2083-03-19	
2.	Write a program that defines a class "Employee" with private data members empId, name, basicSalary, hra, da, grossSalary, tax, and netSalary. Use a parameterized constructor to initialize empId, name, and basicSalary, and member functions to calculate the gross salary and net salary.	2083-03-19	
3.	Write a program that defines a class "Room" with private data members length, breadth, and height (in feet). Implement a default constructor, a parameterized constructor, and a copy constructor. Write member functions to compute the area of the floor and the volume of the room, and a function to display the room's dimensions along with the computed area and volume.	2083-03-19	
4.	Write a program that defines a class "ComplexNumber" with private data members real and imag. Implement a default constructor and a parameterized constructor. Write member functions to add and subtract two complex numbers, where one object is passed as a function argument and a new object is returned as the result. Display the complex numbers in the form a + bi.	2083-03-19	
5.	Write a program that defines a class "Counter" with a private static data member count that keeps track of the total number of objects created. Use a constructor that increments count every time an object is created, and a destructor that decrements count every time an object is destroyed. Implement a static member function to return the current value of count.	2083-03-19	

Lab Index for Lab Sheet 3

SN	Title	Lab Date	Signature
1.	Write a program that defines a class "Counter" with a private data member count. Overload the ++ operator (both prefix and postfix forms) as member functions to increment count by 1, and write a member function to display the current value of count. <i>Hint: [Counter(int c = 0) → constructor that initializes count</i> <i>Counter operator ++ () → prefix version, increments count then returns the object</i> <i>Counter operator ++ (int) → postfix version, returns the old value then increments count</i> <i>displayCount() → prints the current value of count]</i>	2083-03-29	
2.	Write a program that defines a class "Complex" with private data members real and imag. Overload the binary + operator as a member function to add two Complex objects. <i>Hint: [Complex() → default constructor, initializes real and imag to 0</i> <i>Complex(float r, float i) → parameterized constructor</i> <i>Complex operator + (Complex c2) → member function that adds the real parts and imaginary parts of two Complex objects</i> <i>displayComplex() → prints the complex number in the form a + bi]</i>	2083-03-29	
3.	Write a program that defines a class "Distance" with private data members feet and inches. Overload the binary + operator using friend function to add two Distance objects. Handle the case where the resulting inches value is 12 or more by carrying over into feet. <i>Hint: [Distance() → default constructor, initializes feet and inches to 0</i> <i>Distance(int f, int i) → parameterized constructor</i> <i>Distance operator + (Distance d2) → friend function that adds feet and inches, carries over 12 inches into 1 foot</i> <i>displayDistance() → prints the distance in the form feet'-inches"]</i>	2083-03-29	

4.	Write a program that defines a class "Time" with private data members hours and minutes. Overload the binary + operator as a friend function to add two Time objects, and overload the == operator as a friend function to check whether two Time objects are equal. <i>Hint: [friend Time operator + (Time t1, Time t2) → adds hours and minutes, carries over 60 minutes into 1 hour</i> <i>friend bool operator == (Time t1, Time t2) → returns true if hours and minutes of both objects match</i> <i>displayTime() → prints the time in the form hh:mm]</i>	2083-03-29	
----	--	------------	--

Lab Index for Lab Sheet 4

SN	Title	Lab Date	Signature
1.	Write a program that defines a class "Distance" with private data members feet and inches. Write a conversion constructor that converts a basic float value (representing total distance in feet, e.g., 5.75) into a Distance object, splitting it into whole feet and inches. <i>Hint: [Distance(float f) → conversion constructor that takes a float value f (total feet), sets feet = (int)f, and sets inches = (f – feet) * 12</i> <i>displayDistance() → prints the distance in the form feet'-inches"]</i>	2083-03-31	
2.	Write a program that defines a class "Distance" with private data members feet and inches. Overload the type conversion operator to convert a Distance object into a basic float value representing the total distance in feet (i.e., feet + inches/12). <i>Hint: [Distance(int f, int i) → parameterized constructor</i> <i>operator float() → conversion operator (no return type specified, no arguments) that returns feet + (inches / 12.0) as a float</i> <i>Test by assigning a Distance object directly to a float variable, e.g., float d = distObj;]</i>	2083-03-31	
3.	Write a program that defines two classes "Kilometers" and "Miles", where Kilometers has a private data member km, and Miles has a private data member miles. Write a conversion constructor inside the Miles class that converts a Kilometers object into a Miles object (assume 1 km = 0.621371 miles). <i>Hint: [Kilometers(float k) → constructor to initialize a Kilometers object</i> <i>Miles(Kilometers k) → conversion constructor inside the Miles class that computes miles = k.km * 0.621371</i> <i>Use public getter method in Kilometers to access km</i> <i>displayMiles() → prints the distance in the form "Miles: miles"</i> <i>displayKilometers() → prints the distance in the form "Kilometers: km"]</i>	2083-03-31	

4.	Write a program that defines two classes "Dollars" and "Rupees", where Dollars has a private data member amount, and Rupees has a private data member amount. Overload the type conversion operator inside the Dollars class to convert a Dollars object into a Rupees object (assume 1 Dollar = 135 Rupees). <i>Hint: [Dollars(float amt) → parameterized constructor operator Rupees() → conversion operator inside Dollars that returns a Rupees object with amount = amount * 135 displayRupees() → prints the amount in the form "Rs. amount" Rupees must be declared/defined before the Dollars class, or forward declared, since Dollars needs to know about Rupees]</i>	2083-03-31	
----	---	------------	--

Lab Index for Lab Sheet 5

SN	Title	Lab Date	Signature
1.	Create a class Employee that stores the employee ID, name, and basic salary. Derive a class Salary that stores House Rent Allowance (HRA), Dearness Allowance (DA), and Bonus of basic salary. Initialize 20 % , 10% and 5% of basic to HRA, DA and bonus. Calculate the gross salary by adding the basic salary, HRA, DA, and bonus, and apply 3% tax deduction on gross salary to compute net salary. Display the employee details along with the salary details.	2083-04-06	
2.	Create a Student with data members such as student ID, name, and semester, initialized through a parameterized constructor. Derive a class Marks that stores marks in five subjects using its own parameterized constructor. Further derive a class Result that calculates the total, percentage, and division (Distinction, First, Second, Pass, or Fail). Assign the division as Distinction ($\geq 80\%$), First (60–79 %), Second (45–59%), Pass (32%), or Fail , if the percentage is below 32% or the student scores less than 32 marks in any subject. Display the complete student details, marks, total, percentage, and division.	2083-04-06	
3.	Create a base class Product with product ID, product name, and unit price initialized using a constructor. Derive a class Order that stores quantity purchased and calculates the total amount. Further derive a class Discount that applies a discount according to the following rules: i. Total $\geq$ Rs. 50,000 $\rightarrow$ 15% discount ii. Total $\geq$ Rs. 20,000 $\rightarrow$ 10% discount iii. Otherwise $\rightarrow$ No discount Use constructors to initialize all required data members and display the final payable amount	2083-04-06	

Lab Index for Lab Sheet 6

SN	Title	Lab Date	Signature
1.	Write a program that defines a base class "Shape" with a virtual function area() that returns 0. Derive two classes "Circle" and "Rectangle" from Shape that override area() to compute their respective areas. Use a base class pointer to call the overridden function on each derived object and demonstrate late binding. <i>Hint: [virtual float area() → virtual function in Shape, returns 0</i> <i>Circle(float r) → parameterized constructor; overridden area() returns $3.1416 * r * r$</i> <i>Rectangle(float l, float w) → parameterized constructor; overridden area() returns $l * w$</i> <i>Use Shape* ptr = &circleObj; ptr->area(); and Shape* ptr2 = &rectObj; ptr2->area(); to demonstrate late binding through a base class pointer]</i>	2083-04-11	
2.	Write a program that defines an abstract class "Animal" with a pure virtual function makesound(). Derive two classes "Dog" and "Cat" from Animal that implement makesound() differently. Use base class pointers to call the correct overridden function for each derived object. <i>Hint: [virtual void makesound() = 0; → pure virtual function that makes Animal an abstract class</i> <i>Dog::sound() → prints "Dog barks"</i> <i>Cat::sound() → prints "Cat meows"</i> <i>Use Animal* ptr = new Dog(); ptr->makesound(); and Animal* ptr2 = new Cat(); ptr2->makesound(); to call the derived versions</i> <i>Give Animal a virtual destructor so that deleting through a base pointer correctly calls the derived class destructor]</i>	2083-04-11	
3.	Write a program that defines a class "Account" with a private data member balance and a static data member totalAccounts that keeps track of how many Account objects have been created. Use a friend function to add interest to the balances of two different Account objects at once. <i>Hint: [Account(float bal) → parameterized constructor that increments totalAccounts</i> <i>static int totalAccounts; → static data member shared by all objects</i> <i>static int getCount() → static member function that returns totalAccounts</i> <i>friend void addInterest(Account &a1, Account &a2, float rate) → friend function that adds rate% interest to the balance of both a1 and a2</i> <i>displayBalance() → prints "Balance: balance"]</i>	2083-04-11	

